

CS 171 Homework 4, Knight Tour

3. Implementing Knight's Tour Using a Stack. (60 pts) You know the chess piece that looks like a horse? This is called a 'knight.' Recall that the knight can move in the shape of the letter 'L' in any direction; i.e., it can move two squares horizontally followed by one square vertically, or two squares vertically followed by one horizontally. For example, if the Knight is located at the position marked X in the figure on the right, it can move to any of the Y positions.

An interesting question is: can a knight move around the board, visiting each square once, and only once? This is called the 'Knight's Tour' problem. Here is an example table on the right of a 'tour' by a knight on a 3x3 board, starting at the top left corner, where the knight attempts to visit each square exactly once.

	1	6	3
	4		8
	7	2	5

The numbers indicate how many positions the knight has already visited during its tour. For example, the number 3 in the position indexed (0,2) on this 2D array means this was the knight's 3rd step; i.e., the knight's path was $(0,0) \rightarrow (2,1) \rightarrow (0,2)$ and so on. Note that the center position was never visited.

Indeed, it is impossible for a knight to reach the center of a 3x3 chess board, but this shown tour is still valid (for a 3x3 board) since the maximum number of squares was visited on the board, and each position was visited exactly once. Right figure is another example of a valid tour, but this time for a 5x5 board where the knight was able to visit every square on the board, and exactly once.

+	+	+	+	+	+				
	1		10		5		16		25
+	+	+	+	+	+				
	4		17		2		11		6
+	+	+	+	+	+				
	9		20		13		24		15
+	+	+	+	+	+				
	18		3		22		7		12
+	+	+	+	+	+				
	21		8		19		14		23

Goal: Your goal for this question is to write a Java program that searches for a valid knight's tour on a board (whose size is passed as a parameter), starting at the top left corner. Your solution should make the knight visit exactly once the maximum number of positions on the board. Note that there can be several distinct valid tours for a given board. Your job is **NOT** to find every possible valid tour; rather, you should stop at the first valid tour you discover. To get a sense of how many valid board configurations may exist for different board sizes, see "[The Knight's Paths](http://www.behnel.de/knight.html)" table here: <http://www.behnel.de/knight.html>

Data Structures Your solution **must** use a **stack** to keep track of different possible sequences of moves made by the knight, until you discover (and return) a valid tour. This paradigm where you incrementally try different solutions until you identify one that works is called backtracking, since you 'backtrack' from a candidate solution once you realize it doesn't work. Many backtracking problems can be approached using recursion, or iteratively using stacks since they easily help you keep track of your 'most recent' moves. For this question, your solution must utilize a stack and **not rely on recursion**. Use Java's Stack implementation:

<https://docs.oracle.com/javase/8/docs/api/java/util/Stack.html>

Knight's Tour Algorithm: Here is the pseudocode for the algorithm you should implement to find a valid knight's tour (some of the details are left for you to fill in). You will use a stack to push the next valid board when the knight makes a legal move, and to pop the next board at each iteration. This helps you search through different sequences of movements. Think carefully about when a move is 'valid' and how you can ensure that your implementation terminates.

```
Create a search list (as a stack) with the initial board to explore, which will have the knight at the starting position (0,0)
while the stack is not empty do
    remove the next board from the search list
    if the knight has visited all squares on this board then
        a solution was found, terminate and return this board
    else
        explore all possible (legal) next moves for the knight (from its current position), while pushing the new possible board(s)
        to the search list
    end if
end while
if the list is empty and the method has not returned yet then
    all possible sequences of knight movements have been explored, return the current board
end if
```

Starter Code Description

(1) **The Board (KnightBoard.java):** This class models a chess board as a 2- dimensional array of integers, and has data members that help you keep track of the knight's movements on it, like its current (x,y) coordinates, number of moves made so far, etc. It also includes other important helper methods. The main method in this class will also give you hints on how to use the board's methods. Start by reading the code in KnightBoard.java carefully, including all of our comments there.

(2) **The Knight's Tour (KnightTour.java):** This is the only file you need to edit and submit. Your program must use a stack of KnightBoard objects to search for (and return) a valid knight's tour. All of this will happen inside the method tour(int n). You can create other helper methods in this class if needed, but make sure they are all declared static.

Note: While exploring a tour, it is your job to move the knight in legal positions; i.e., the knight begins at (0,0), with one possible next move being (1,2) and another possible one being (2,1). Make sure you explore all possible legal moves for a knight, but note that the method move in class KnightBoard also includes some checks to validate a move; read it carefully before you design your solution.

Getting Started

1. Your starting point is KnightBoard.java. Read this class carefully and run the test cases in its main method make sure you understand how the board works.
2. Now read KnightTour.java carefully and implement the method tour(int n), while referring to this handout's detailed instructions. To test your solution, run the class KnightTour. The main method will initialize a 3x3 board for you by default. Verify that the output is correct.
3. Test your solution for larger boards by passing 'n' as a command line argument to class KnightTour. (To input a command line argument in Eclipse or IntelliJ, you can click "Run" –

“Run Configurations”- “Arguments” and then type in the argument in the \Program Arguments” box.)

Here's an example of how a program run might look like, when passing 5 as a command line argument:

```
$ java KnightTour 5
Possible Knight Tour with max #squares visited in this (5x5) board:
+-----+
| 1|10| 5|16|25|
+-----+
| 4|17| 2|11| 6|
+-----+
| 9|20|13|24|15|
+-----+
|18| 3|22| 7|12|
+-----+
|21| 8|19|14|23|
+-----+
(Time to find this solution = 53.451873 milliseconds)
```

4. Submit **only KnightTour.java** on Canvas for this question.

5. Debugging Tip: You can call the method printChessBoard from class KnightBoard to help you visualize the board and debug intermediate steps in your solution. But please remember to remove or comment out any print calls you added to the code, since these can affect the performance of your program significantly (just leave the one existing call to printChessBoard in KnightTour's main method).

Starter code:

```
public class KnightBoard {

    // NOTE: Do NOT modify anything in this class. The only
    // method you can edit is the main method, for your own
    // testing purposes.

    private int size; // the board's width and height
    private int moveCount; // number of positions visited by the Knight so far
    private int currentX, currentY; // Knight's current (x,y) position

    public int[][] board; // the chessboard: spaces that have never been
visited
                                // are marked with -1. Visited spaces will contain positive
                                // integers, indicating the number of moves made so far
                                // by the Knight (i.e. 1 refers to the starting position)

    /** Constructor to initialize the board
     * @param n the width/height of the board */
    public KnightBoard(int n){
        this.size = n;
        board = new int[size][size];
    }
}
```

```

    for (int i=0; i < board.length; i++) {
        for (int j=0; j < board.length; j++)
            board[i][j] = -1; // -1 indicates a non-visited position
    }
    // Knight starts at position (0,0), i.e. top-left corner
    currentX= 0;
    currentY= 0;
    moveCount = 1;
    board[currentX][currentY] = moveCount; // mark the first move on the
board
}

/** Helper method for printing the chessboard to standard output */
public void printChessBoard() {
    for ( int i = 0; i < this.size; i++ ){
        for ( int h = 0; h < this.size; h++ )
            System.out.print("+--");
        System.out.println("+");
        for ( int j = 0; j < this.size; j++ ){
            System.out.print("|");
            if ( board[i][j] == -1 )
                System.out.print(" ");
            else if ( board[i][j] < 10 )
                System.out.print(" " + board[i][j]);
            else System.out.print(board[i][j]);
        }
        System.out.println("|");
    }
    for ( int h = 0; h < this.size; h++ )
        System.out.print("+--");
    System.out.println("+");
}

/** Helper method that creates an exact copy of the current board
* @return the newly created board */
public KnightBoard copyBoard(){
    KnightBoard copy = new KnightBoard(this.size);
    copy.setMoveCount(this.moveCount);
    copy.setCurrentX(this.currentX);
    copy.setCurrentY(this.currentY);
    for ( int i = 0; i < this.size; i++ )
        for ( int j = 0; j != this.size; j++ )
            copy.board[i][j] = this.board[i][j];
    return copy;
}

// The board's getters and setters methods:
public int getMoveCount(){ return moveCount; }
public int getCurrentX(){ return currentX; }
public int getCurrentY(){ return currentY; }
public void setMoveCount(int count){ moveCount = count; }
public void setCurrentX(int x) { currentX = x;}
public void setCurrentY(int y){ currentY = y;}

/** Place the Knight to the new position passed as a parameter.
* NOTE: This method assumes that the caller is passing a valid position,
* with respect to how a Knight moves on a chessboard. However, this

```

```

    * method additionally checks if the new position has not been visited
before
    * and that it is within the board's valid boundaries (borders).
    * @param newX - the x coordinate of the new position
    * @param newY - the y coordinate of the new position
    * @return true, if the Knight made the move successfully.
    * return false, if the move is outside of board boundaries or
    * if the new position has already been visited. */
    public boolean move(int newX, int newY) {
        boolean valid;
        // Check if new position is on the board and has never been visited:
        valid = newX >= 0 && newX < size && newY >= 0 && newY < size
            && board[newX][newY] == -1;

        // NOTE: Again, this method doesn't check if the new position
        // is valid w.r.t. how the Knight moves on a chessboard; you
        // should handle that in your KnightTour implementation!

        // Moving the Knight entails: updating current coordinates,
        // updating moveCount, and marking the new position with moveCount
        if ( valid ){
            this.currentX = newX;
            this.currentY = newY;
            this.moveCount = this.moveCount + 1;
            this.board[currentX][currentY] = this.moveCount;
        }
        return valid;
    }
}

// Main method for testing the logic of some KnightBoard's methods
// NOTE: You can modify this main method on your computer for testing
// purposes and to better understand how KnightBoard works
// (but do NOT modify any other method in this class!)
public static void main(String[] args){
    KnightBoard b = new KnightBoard(4);
    System.out.println("Show the initial board with the Knight's starting
position:");
    b.printChessBoard();
    System.out.println("\nIs the move 1,2 valid? " + b.move(1,2));
    b.printChessBoard();
    System.out.println("\nIs the move 0,0 valid? " + b.move(0,0));
    b.printChessBoard();
    System.out.println("\nIs the move 1,2 valid? " + b.move(1,2));
    b.printChessBoard();
    System.out.println("\nIs the move 3,1 valid? " + b.move(3,1));
    b.printChessBoard();
    System.out.println("\nCurrent Knight position: " + b.getCurrentX() + ","
+ b.getCurrentY());
}
}

```

```

import java.util.Stack;

// Do you know the chess piece that looks like a horse? It's called a
// "Knight",
// and the "Knight's Tour" is a famous problem where you want the Knight to
// move
// around a chess board such that it visits every position on the board
// exactly once.
// Recall that the Knight can move in the shape of the letter "L" in any
// direction
// IMPORTANT: See our A2 handout for examples and more detailed
// instructions/hints!

public class KnightTour {
    // TODO: Implement and print a Knight's Tour solution, using
    // the chess board defined in class KnightBoard. Your solution must
    // use a Stack to keep track of different possible sequences of moves
    // for the Knight, in order to discover and return a valid tour.

    // NOTE: There can be several distinct valid tours; your job is to find
    // and print only one valid tour (the first valid one you discover).
    // Please do NOT attempt to discover all possible valid tours! To get
    // a sense of how many valid board configurations exist for different
    // board sizes, see "The Knight's Paths" table here:
    // http://www.behnel.de/knight.html

    /** tour method is where you add your code for implementing
     * a Knight Tour's solution for an n*n chess board
     * @param n size of the board
     * @return KnightBoard object with a valid Knight Tour
     */
    public static KnightBoard tour(int n){
        // Your solution must utilize the stack "candidates" below,
        // to keep track of different possible sequences of Knight moves
        Stack<KnightBoard> candidates = new Stack<KnightBoard>();
        KnightBoard kb = new KnightBoard(n); // create initial board of size
n*n
        candidates.push(kb); // push the initial board onto the stack

        // TODO: ..... Add your solution code here ..... //

        return null; //TODO: return a KnightBoard object with a valid Knight
Tour
    }

    // Do NOT modify this main method. If you need to add code for
    // testing your solution, just make sure to comment it out before
    submission
    public static void main(String[] args) {
        int n = 3; // default board size if user didn't specify
        // pass in parameter n from command line
        if (args.length == 1) {
            n = Integer.parseInt(args[0].trim());
            if (n < 3) {
                System.out.println("Incorrect parameter (n must be >= 3)");
                System.exit(-1);
            }
        }
    }
}

```

```

    }
}
long startTime = System.nanoTime();
KnightBoard winner = KnightTour.tour(n);
long endTime = System.nanoTime();
double delta = (endTime - startTime) / 1e6;
// Display the solution you discovered:
System.out.println("\nPossible Knight Tour with max #squares visited in
this (" + n + "x" + n + ") board:");
winner.printChessBoard();
System.out.println("\n(Time to find this solution = " + delta + "
milliseconds)");
}
}

```

Sample running time:

\$ java KnightTour 3

Possible Knight Tour with max #squares visited in this (3x3) board:

```

+--+--+--+
| 1| 6| 3|
+--+--+--+
| 4| | 8|
+--+--+--+
| 7| 2| 5|
+--+--+--+

```

(Time to find this solution = 0.771277 milliseconds)

\$ java KnightTour 4

Possible Knight Tour with max #squares visited in this (4x4) board:

```

+--+--+--+--+

```

| 1| 8|13|10|

+--+--+--+--+

|14|11| 4| 7|

+--+--+--+--+

| 5| 2| 9|12|

+--+--+--+--+

| |15| 6| 3|

+--+--+--+--+

(Time to find this solution = 10.917376 milliseconds)

\$ java KnightTour 5

Possible Knight Tour with max #squares visited in this (5x5) board:

+--+--+--+--+

| 1|10| 5|16|25|

+--+--+--+--+

| 4|17| 2|11| 6|

+--+--+--+--+

| 9|20|13|24|15|

+--+--+--+--+

|18| 3|22| 7|12|

+--+--+--+--+

|21| 8|19|14|23|


```
+--+--+--+--+--+
```

(Time to find this solution = 55.889333 milliseconds)

```
$ java KnightTour 6
```

Possible Knight Tour with max #squares visited in this (6x6) board:

```
+--+--+--+--+--+
```

```
| 1|12|35| 8|25|14|
```

```
+--+--+--+--+--+
```

```
|36| 7| 2|13|34|23|
```

```
+--+--+--+--+--+
```

```
|11|30| 9|24|15|26|
```

```
+--+--+--+--+--+
```

```
| 6| 3|18|31|22|33|
```

```
+--+--+--+--+--+
```

```
|29|10| 5|20|27|16|
```

```
+--+--+--+--+--+
```

```
| 4|19|28|17|32|21|
```

```
+--+--+--+--+--+
```

(Time to find this solution = 236.791035 milliseconds)

```
$ java KnightTour 7
```

Possible Knight Tour with max #squares visited in this (7x7) board:

+--+--+--+--+--+--+

| 1|30| 9|24|43|28|17|

+--+--+--+--+--+--+

|10|23| 2|29|16|25|42|

+--+--+--+--+--+--+

|31| 8|11|44|41|18|27|

+--+--+--+--+--+--+

|22| 3|32|15|26|45|40|

+--+--+--+--+--+--+

| 7|12| 5|38|19|48|35|

+--+--+--+--+--+--+

| 4|21|14|33|36|39|46|

+--+--+--+--+--+--+

|13| 6|37|20|47|34|49|

+--+--+--+--+--+--+

(Time to find this solution = 90.314049 milliseconds)

\$ java KnightTour 8

Possible Knight Tour with max #squares visited in this (8x8) board:

+--+--+--+--+--+--+

| 1|24| 9|62|39|26|45|60|

+--+--+--+--+--+--+--+--+

|10|63| 2|25|44|61|38|27|

+--+--+--+--+--+--+--+--+

|23| 8|11|40|21|42|59|46|

+--+--+--+--+--+--+--+--+

|64| 3|22|43|58|47|28|37|

+--+--+--+--+--+--+--+--+

| 7|12| 5|20|41|36|57|48|

+--+--+--+--+--+--+--+--+

| 4|17|14|31|50|29|54|35|

+--+--+--+--+--+--+--+--+

|13| 6|19|16|33|52|49|56|

+--+--+--+--+--+--+--+--+

|18|15|32|51|30|55|34|53|

+--+--+--+--+--+--+--+--+

(Time to find this solution = 94947.098818 milliseconds)